```
#include <stdio.h>

struct Process {
   int pid;
   int at;     // Arrival Time
   int bt;     // Burst Time
   int rt;     // Remaining Time
   int ct;     // Completion Time
   int tat;    // Turnaround Time
   int wt;     // Waiting Time
   int started;
};

int main() {

   // Process data: PID, Arrival Time, Burst Time
   struct Process p[3] = {
      {1, 0, 10},
      {2, 2, 20},
      {3, 6, 30}
   };

   int n = 3;
   int time = 0;
   int completed = 0;
   int context_switch = 0;
   int prev = -1;
   int total_wt = 0, total_tat = 0;

   printf("========================================\n");
   printf("   SJF PREEMPTIVE SCHEDULING ALGORITHM   \n");
   printf("========================================\n\n");

   // Initialize remaining time
   for(int i = 0; i < n; i++) {
      p[i].rt = p[i].bt;
      p[i].started = 0;
   }

   // Display input processes
   printf("INPUT PROCESSES:\n");
   printf("PID\tArrival\tBurst\n");
   printf("-------------------\n");
   for(int i = 0; i < n; i++) {
      printf("P%d\t%d\t%d\n", p[i].pid, p[i].at, p[i].bt);
   }

   printf("\nEXECUTION SEQUENCE (Gantt Chart):\n");
```

```
printf("Time\tProcess\tRemaining\n");
printf("----\t-------\t---------\n");

// SJF Preemptive Algorithm
while(completed < n) {

    int idx = -1;
    int min_rt = 10000;

    // Find process with shortest remaining time
    for(int i = 0; i < n; i++) {
        if(p[i].at <= time && p[i].rt > 0) {
            if(p[i].rt < min_rt) {
                min_rt = p[i].rt;
                idx = i;
            }
        }
    }

    if(idx != -1) {

        // Context switch check
        if(prev != -1 && prev != idx) {
            context_switch++;
        }
        prev = idx;

        printf("%d\tP%d\t%d\n", time, p[idx].pid, p[idx].rt);

        // Execute for 1 unit time
        p[idx].rt--;
        time++;

        // Process completion
        if(p[idx].rt == 0) {
            p[idx].ct = time;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;

            total_wt += p[idx].wt;
            total_tat += p[idx].tat;

            completed++;
        }
    }
    else {
        printf("%d\tIdle\t-\n", time);
        time++;
```

```
        }
    }

    printf("\n==========================================\n");
    printf("          RESULTS & CALCULATIONS        \n");
    printf("==========================================\n\n");

    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
    printf("------------------------------------------\n");
    for(int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].pid, p[i].at, p[i].bt,
               p[i].ct, p[i].tat, p[i].wt);
    }

    printf("\nPERFORMANCE METRICS:\n");
    printf("------------------------------------------\n");
    printf("Total Context Switches: %d\n", context_switch);
    printf("Total Waiting Time: %d ns\n", total_wt);
    printf("Total Turnaround Time: %d ns\n", total_tat);
    printf("Average Waiting Time: %.2f ns\n", (float)total_wt / n);
    printf("Average Turnaround Time: %.2f ns\n", (float)total_tat / n);

    printf("\n==========================================\n");
    printf("       VERIFICATION CALCULATIONS        \n");
    printf("==========================================\n\n");

    printf("Manual Calculations:\n");
    printf("P1: CT = 10, TAT = 10-0 = 10, WT = 10-10 = 0\n");
    printf("P2: CT = 30, TAT = 30-2 = 28, WT = 28-20 = 8\n");
    printf("P3: CT = 60, TAT = 60-6 = 54, WT = 54-30 = 24\n");
    printf("Avg WT = (0+8+24)/3 = 32/3 = 10.67 ns\n");

    return 0;
}
```

Output

Clear

```
==========================================
   SJF PREEMPTIVE SCHEDULING ALGORITHM
==========================================

INPUT PROCESSES:
PID Arrival Burst
-------------------
P1  0   10
P2  2   20
P3  6   30

EXECUTION SEQUENCE (Gantt Chart):
Time    Process Remaining
----    ------- ---------
0   P1  10
1   P1  9
2   P1  8
3   P1  7
4   P1  6
5   P1  5
6   P1  4
7   P1  3
8   P1  2
9   P1  1
10  P2  20
11  P2  19
```

Output

Clear

```
10  P2  20
11  P2  19
12  P2  18
13  P2  17
14  P2  16
15  P2  15
16  P2  14
17  P2  13
18  P2  12
19  P2  11
20  P2  10
21  P2  9
22  P2  8
23  P2  7
24  P2  6
25  P2  5
26  P2  4
27  P2  3
28  P2  2
29  P2  1
30  P3  30
31  P3  29
32  P3  28
33  P3  27
34  P3  26
35  P3  25
```

```
25  P2  5
26  P2  4
27  P2  3
28  P2  2
29  P2  1
30  P3  30
31  P3  29
32  P3  28
33  P3  27
34  P3  26
35  P3  25
36  P3  24
37  P3  23
38  P3  22
39  P3  21
40  P3  20
41  P3  19
42  P3  18
43  P3  17
44  P3  16
45  P3  15
46  P3  14
47  P3  13
48  P3  12
49  P3  11
50  P3  10
51  P3  9
52  P3  8
53  P3  7
54  P3  6
55  P3  5
56  P3  4
57  P3  3
```

```c

struct Process {
    int pid;
    int at;     // Arrival Time
    int bt;     // Burst Time
    int rt;     // Remaining Time
    int ct;     // Completion Time
    int tat;    // Turnaround Time
    int wt;     // Waiting Time
    int started;
};

int main() {

    // Process data: PID, Arrival Time, Burst Time
    struct Process p[3] = {
        {1, 0, 10},
        {2, 2, 20},
        {3, 6, 30}
    };

    int n = 3;
    int time = 0;
    int completed = 0;
    int context_switch = 0;
    int prev = -1;
    int total_wt = 0, total_tat = 0;

    printf("========================================\n");
    printf("   SJF PREEMPTIVE SCHEDULING ALGORITHM   \n");
    printf("========================================\n\n");

    // Initialize remaining time
```

```
58  P3  2
59  P3  1

========================================
           RESULTS & CALCULATIONS
========================================

PID AT  BT  CT  TAT WT
----------------------------------------
P1  0   10  10  10  0
P2  2   20  30  28  8
P3  6   30  60  54  24

PERFORMANCE METRICS:
----------------------------------------
Total Context Switches: 2
Total Waiting Time: 32 ns
Total Turnaround Time: 92 ns
Average Waiting Time: 10.67 ns
Average Turnaround Time: 30.67 ns

========================================
        VERIFICATION CALCULATIONS
========================================

Manual Calculations:
P1: CT = 10, TAT = 10-0 = 10, WT = 10-10 = 0
P2: CT = 30, TAT = 30-2 = 28, WT = 28-20 = 8
P3: CT = 60, TAT = 60-6 = 54, WT = 54-30 = 24
Avg WT = (0+8+24)/3 = 32/3 = 10.67 ns


=== Code Execution Successful ===
```